

07. live 系 node 構成ビルダ

solar_ws0129-main

2026-07-29

対象

項目	内容
ファイル	mpc_solarcar/live_launch.py
種別	Python
区分	launch helper
役割	profile を読み、live と live_wifi で共通に使う node 群を Python から組み立てる。
起動文脈	launch/solar_race_live*.launch.py から呼ばれる。

このファイルを読む理由

profile を読み、live と live_wifi で共通に使う node 群を Python から組み立てる。

- forecast CSV の raw/corrected パスを用意する。
- mpc_node、logger、dashboard、preflight などの Node action を返す。
- use_wifi の有無で planner が読む forecast CSV を切り替える。

呼び出し関係

どこから呼ばれるか

- launch/solar_race_live.launch.py
- launch/solar_race_live_wifi.launch.py

次に読むと理解が進むファイル

- mpc_solarcar/solar_profile.py
- mpc_solarcar/mpc_node.py
- mpc_solarcar/speed_command_bridge_node.py

同一リポジトリ内の主要依存

- mpc_solarcar/solar_profile.py

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

代表コード断片

```
if use_wifi and bool(wind_cfg.get("enabled", True)):
    mpc_forecast_csv = corrected_forecast_csv

nodes = [
    Node(
        package="mpc_solarcar",
        executable="solar_preflight_node",
        name="solar_preflight_node",
        parameters=[
            {
                "require_speed": True,
                "require_distance": True,
                "require_battery": True,
                "require_planner": True,
                **preflight_cfg,
```

主要構造の読み取り

主要関数は `cfg_path`, `live_forecast_paths`, `build_live_nodes`。launch から起動する Node action は 9 件。

主要クラス・関数

行	種別	名前	補足
11	function	<code>cfg_path</code>	args: <code>profile_path</code> , <code>raw</code>
15	function	<code>_drop_keys</code>	args: <code>payload</code>
20	function	<code>live_forecast_paths</code>	args: <code>profile_path</code> , <code>cfg</code>
39	function	<code>build_live_nodes</code>	args: <code>profile_path</code> , <code>cfg</code>

ファイルを上から読んだときの定義順

行	内容
11	関数 <code>cfg_path</code> を定義する。
15	関数 <code>_drop_keys</code> を定義する。
20	関数 <code>live_forecast_paths</code> を定義する。
39	関数 <code>build_live_nodes</code> を定義する。

import 群の詳細

行	import 文	このファイルで読む理由
---	----------	-------------

1	<code>from__future_ _importannotations</code>	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
3	<code>importshutil</code>	shutil モジュールを利用するため。このファイル内での主な使用位置は L35。
4	<code>frompathlibimportPath</code>	ファイルやディレクトリを安全に扱うため。このファイル内での主な使用位置は L32, L33。
6	<code>fromlaunch_ros. actionsimportNode</code>	ROS 2 ノード起動 action を launch から記述するため。このファイル内での主な使用位置は L58, L72, L102, L113, L131, L140, L142, L145, ...。
8	<code>from.solar_ profileimportget_path,get_ section,resolve_relative_ path</code>	profile YAML 読込と検証から get_path, get_section, resolve_relative_path を読み込み、このファイルの内部処理を分担させるため。実体ファイルは mpc_solarcar/solar_profile.py。このファイル内での主な使用位置は L12, L21, L22, L23, L24, L40, L41, L42, ...。

関数・クラスを上から順に解説

L11 関数 `cfg_path`

- 定義: `cfg_path(profile_path, raw)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `resolve_relative_path`, `str`, `strip`
- 戻り値の要点: `resolve_relative_path(profile_path.parent, raw) if str(raw or "").strip() else ""`

1. `resolve_relative_path(profile_path.parent, raw) if str(raw or "").strip() else ""` を返す。

L15 関数 `_drop_keys`

- 定義: `_drop_keys(payload, *keys)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `items`, `set`
- 戻り値の要点: `{key: value for key, value in (payload or {}).items() if key not in blocked}`

1. `blocked` に `set(keys)` の結果を代入する。
2. `{key: value for key, value in (payload or {}).items() if key not in blocked}` を返す。

L20 関数 `live_forecast_paths`

- 定義: `live_forecast_paths(profile_path, cfg)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Path`, `cfg_path`, `copy2`, `exists`, `get`, `get_path`, `get_section`, `makedirs`, `str`
- 戻り値の要点: `(base_path, raw_path, corrected_path)`

1. live_cfg に get_section(cfg, 'live') の結果を代入する。
2. weather_cfg に get_section(live_cfg, 'weather') の結果を代入する。
3. wind_cfg に get_section(live_cfg, 'wind_model') の結果を代入する。
4. base_path に get_path(cfg, profile_path, 'forecast_csv') の結果を代入する。
5. raw_path に cfg_path(profile_path, str(weather_cfg.get('raw_forecast_csv', ''))) の結果を代入する。
6. 条件 not raw_path を判定し、真なら内部処理を行う。
7. corrected_path に cfg_path(profile_path, str(wind_cfg.get('corrected_forecast_csv', ''))) の結果を代入する。
8. 条件 not corrected_path を判定し、真なら内部処理を行う。
9. (raw_path, corrected_path) を順に走査し、各要素を runtime_path に入れて処理する。
10. (base_path, raw_path, corrected_path) を返す。

L39 関数 build_live_nodes

- 定義: build_live_nodes(profile_path, cfg, use_wifi)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: Node, _drop_keys, append, bool, cfg_path, float, get, get_path, get_section, int, live_forecast_paths, str
- 戻り値の要点: nodes

1. runtime_cfg に get_section(cfg, 'runtime') の結果を代入する。
2. logging_cfg に get_section(cfg, 'logging') の結果を代入する。
3. live_cfg に get_section(cfg, 'live') の結果を代入する。
4. weather_cfg に get_section(live_cfg, 'weather') の結果を代入する。
5. autocal_cfg に get_section(live_cfg, 'autocal') の結果を代入する。
6. command_cfg に get_section(live_cfg, 'command_bridge') の結果を代入する。
7. distance_cfg に get_section(live_cfg, 'distance') の結果を代入する。
8. grade_cfg に get_section(live_cfg, 'grade') の結果を代入する。
9. wind_cfg に get_section(live_cfg, 'wind_model') の結果を代入する。
10. live_logging_cfg に get_section(live_cfg, 'logging') の結果を代入する。

launch から起動するノード

行	executable	package / name
58	solar_preflight_node	package=mpc_solarcar, name=solar_preflight_node
72	mpc_node	package=mpc_solarcar, name=mpc_node
102	dashboard_node	package=mpc_solarcar, name=dashboard_node
113	solar_logger_node	package=mpc_solarcar, name=solar_logger_node
131	speed_command_bridge_node	package=mpc_solarcar, name=speed_command_bridge_node

140	distance_node	package=mpc_solarcar, name=distance_node
142	grade_node	package=mpc_solarcar, name=grade_node
145	weather_fetch_node	package=mpc_solarcar, name=weather_fetch_node
168	solar_autocal_node	package=mpc_solarcar, name=solar_autocal_node

処理の流れ

1. ソース中の主要関数を通じて処理を進める。

このファイルの位置づけ

この資料群では、`mpc_solarcar/live_launch.py` を **launch helper** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。